

FULL LEGAL NAME	LOCATION (COUNTRY)	EMAIL ADDRESS	MARK X FOR ANY NON-CONTRIBUTING MEMBER
Miklos Lammel	Budapest, Hungary	lammelm@gmail.com	
Vikas Gautam	Bangalore, India	vikasgtm@gmail.com	
Gaurav Sharma	Gurgaon, India	ggsharma9555@gmail.com	

Statement of integrity: By typing the names of all group members in the text boxes below, you confirm that the assignment submitted is original work produced by the group (excluding any non-contributing members identified with an “X” above).

Team member 1	Miklos Lammel
Team member 2	Vikas Gautam
Team member 3	Gaurav Sharma

Use the box below to explain any attempts to reach out to a non-contributing member. Type (N/A) if all members contributed.

Note: You may be required to provide proof of your outreach to non-contributing members upon request.

--

Introduction

In this project, we explored some of the Deep Learning learning methods particularly RNN. We have downloaded the data from finance for

1. Equity → SPDR S&P500 ETF. Symbol: "SPY"
2. Fixed Income → iShares 20+ Year Treasury Bond ETF. Symbol: "TLT"
3. Cash-like → iShares 1-3 Year Treasury Bond ETF. Symbol: "SHY"
4. Precious metals (Gold) □ SPDR Gold Shares. Symbol: "GLD"
5. Crude Oil → Invesco DB Oil Fund. Symbol: "DBO"
6. Single stock for reference → Tesla: "TSLA"

For he period start='2010-01-01', end='2023-12-31'

STEP 1 - Exploratory Data Analysis

Downloading the data and chosen timeframe

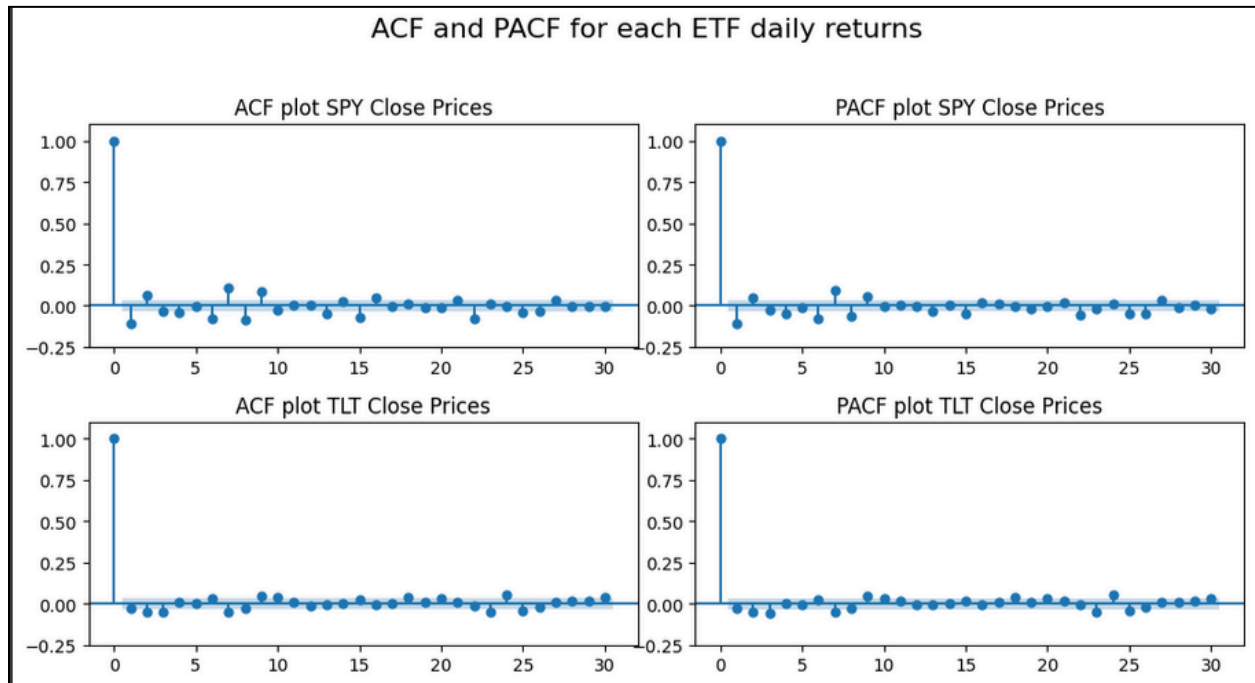
For our exploratory data analysis we have downloaded the 5 given assets from 2011-01-01 to 2024-12-31 and added the Tesla stock for reference, since that was what we have used in our last assignment. This was only for us to double check that our results were consistent. Also the timeframe was chosen, so that we have a full dataset for Tesla, since they went public on the 29th of June, 2010. For our implementation we did stick with the initially decided 2010-01-01 to 2023-12-31 timeframe.

Exploratory data analysis

The exploratory analysis considered the checking of stationarity, seasonality, normality of the distribution of returns, covariance and correlation.

Stationarity

None of the downloaded time series are stationary by default but the **daily returns time series are reasonably stationary** (relative differencing by applying the pct_change function). We investigated the stationarity of the given time series and their differenced version both **numerically** and **visually**. As a result we have lots of plot, so here we only provide a subset of the plots with an explanation. We have created the ACF and PACF plots for both the original close price time series as well for the daily return (differenced) time series. Here in the figure below one can see the of the ACF and PACF plots of the daily returns of the first two assets (the rest of the plots are in the notebook)

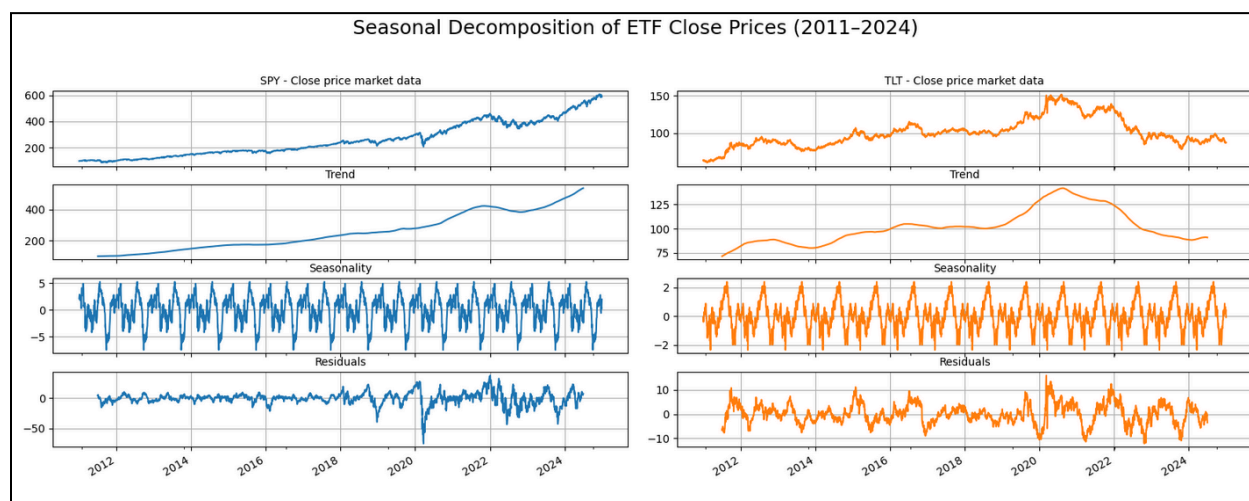


The stationarity of both the original close price time series and the daily returns (differenced time series) have been examined **numerically** also with Augmented Dickey-Fuller (**ADF**) and with Kwiatkowski–Phillips–Schmidt–Shin (KPSS) tests. According to the results both methods have confirmed that the **daily return time series can be considered stationary**, whereas the original non-differenced close-price time series is non-stationary for all assets.

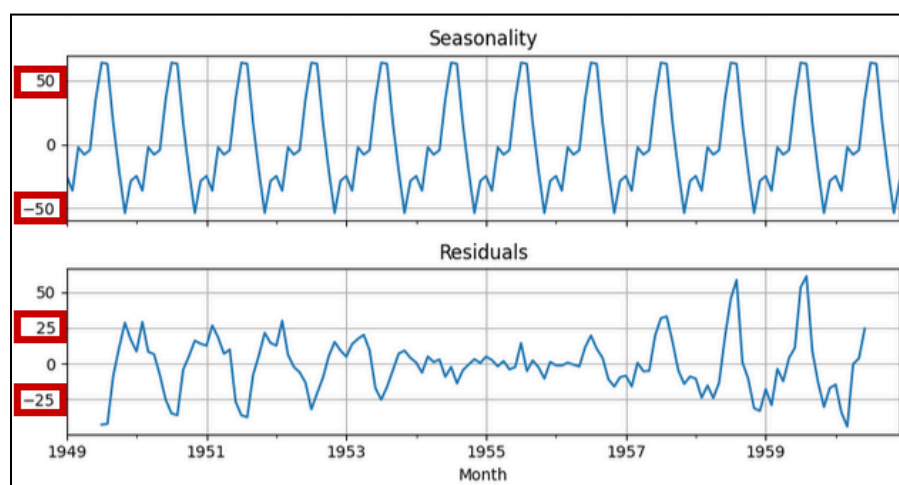
Seasonality

Visually there is no identifiable seasonality in the time series plots of the closing prices, so just by looking at them one would not assume any seasonality.

We explored the question of seasonality also numerically/algorithmically by using the Python Statsmodel library's "seasonal_decompose()" function. It found some very **weak seasonality** in the examined data, but the relevance of the result is questionable because the residual components are one order of magnitude larger than the seasonality component. For comparison we examined two other datasets with known seasonality: The airline passenger dataset, and here we got the expected result, with good, well pronounced seasonality. We also examined the New York daily temperature values, where the seasonal_decompose function also delivered good results and showed the yearly expected seasonality. In both cases the seasonality component was significantly larger than in any of the assets we investigated.



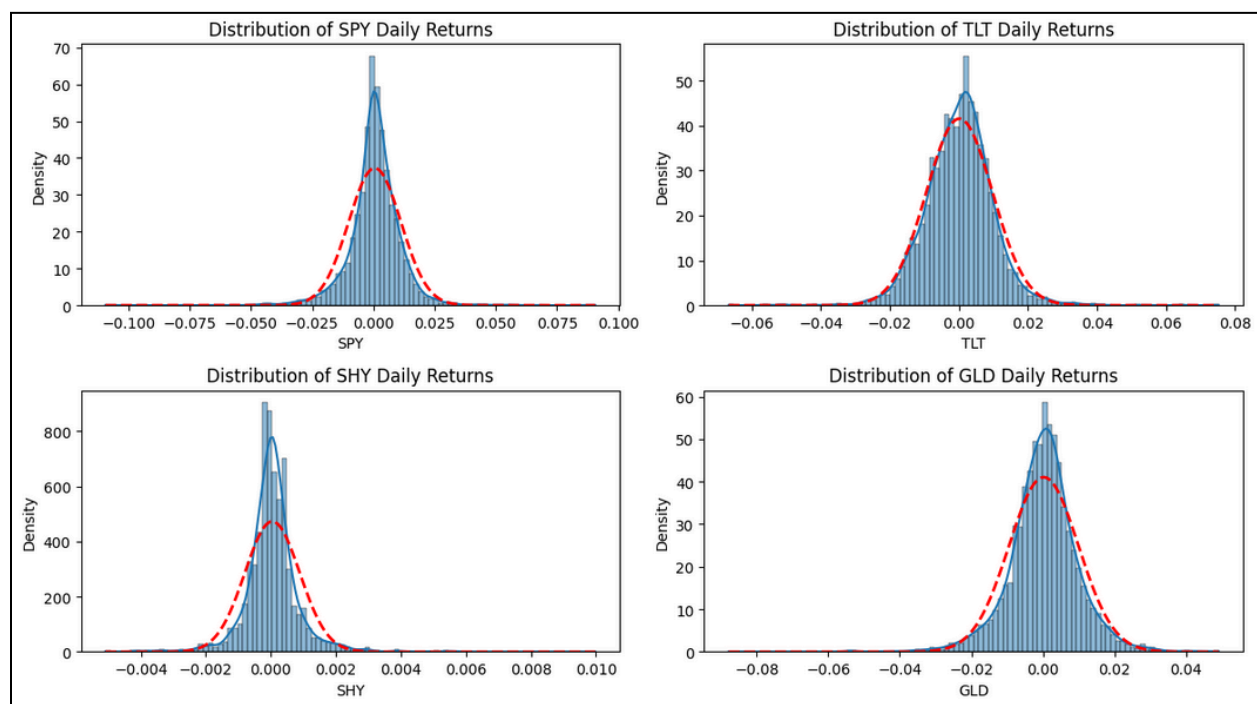
For reference the proper seasonality should have a scale similar or preferably larger than the residuals, like for example in the air passenger dataset below.



Normality of the distribution of returns

We have performed the normality check of the distribution for the daily returns in multiple ways. The **distribution of daily returns** is **non-normal** for all the investigated assets. All of the individual ETFs (and as an extra the Tesla stock - just to satisfy our own curiosity) were both visually and numerically examined.

For the visual representation we have plotted the histogram of the daily returns and overlying the Seaborn's kernel density estimate (kde=True). And on top of this we have plotted the normal distribution curve. This way one can tell immediately that all of the datasets have some kurtosis and a little skewness. TLT has the least amount of kurtosis. The figure below is a cropped version, the full plot with all the assets can be seen in the notebook.

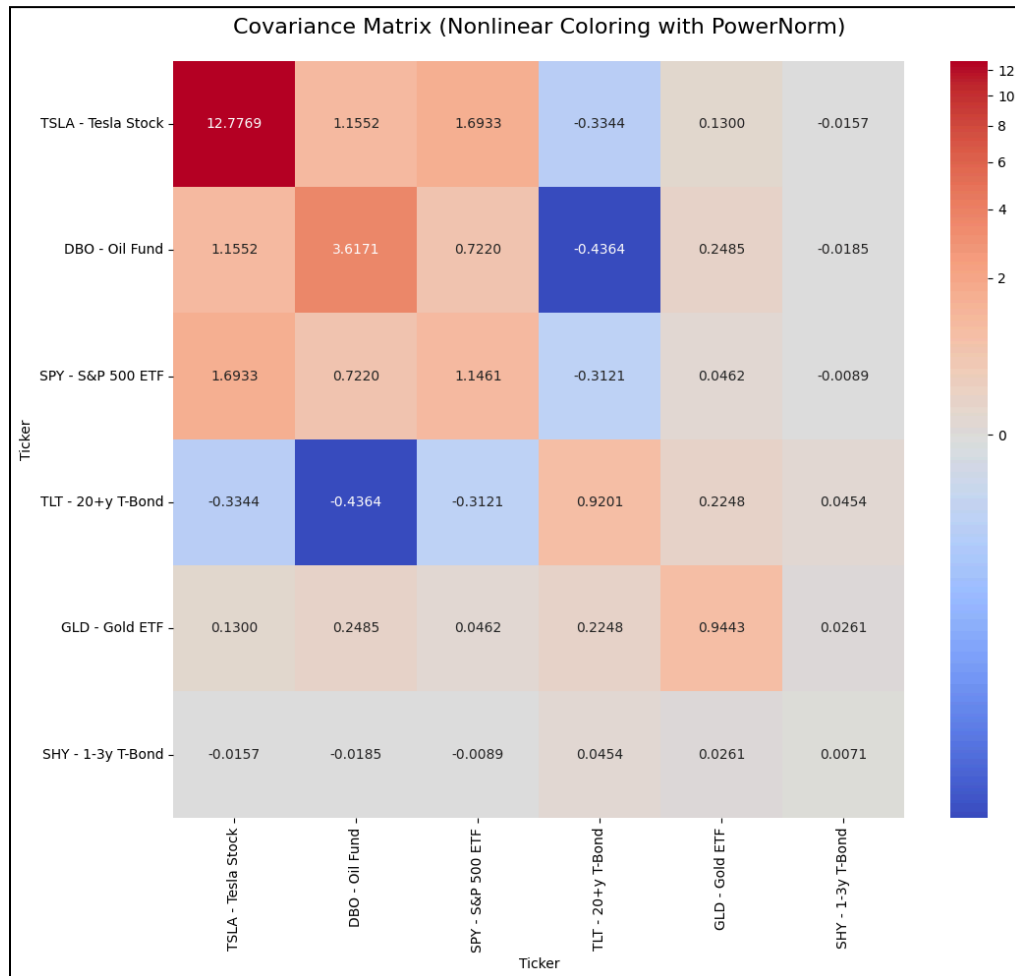


We have also looked at the numerical values of kurtosis and skewness, and they confirm what the plots show us.

The Shapiro-Wilk, Kolmogorov-Smirnov tests also confirmed the lack of normality.

Covariance

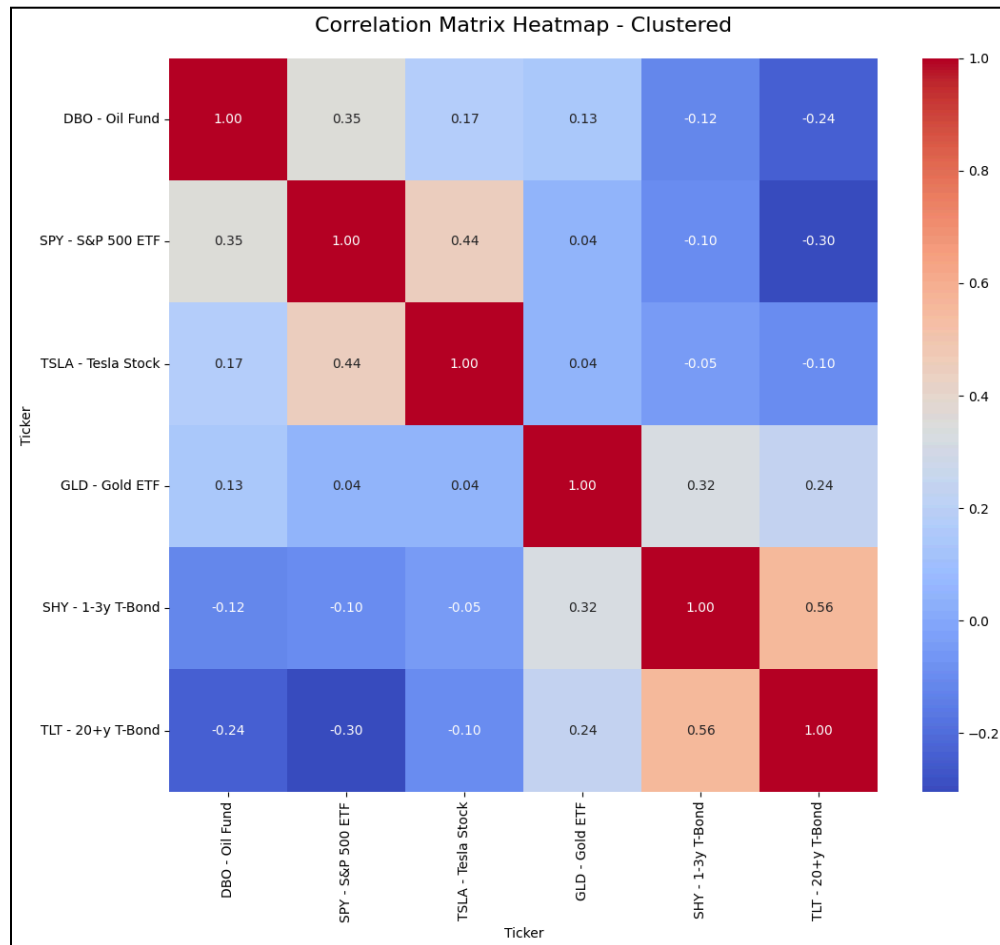
The covariance between the various asset daily returns can be seen below. This plot was created with an order provided by clustering and using a non-linear color scale for better visibility.



Correlation

The correlation matrix between the various asset daily returns can be seen below. This plot was created with an order provided by hierarchical clustering.

Noticeable is the correlation between the Tesla stock (which we added for reference) and SP500 and between the bonds. Interesting is the negative correlation between the Oil Fund and the Bonds.



STEP 2

1. Feature Engineering:

We have developed some new features based on the features we got from the Yahoo Finance downloaded data.

The dataset originally has Open, Low, High, Close and Volume.

For e.g. for Ticker: 'SHY', we have a "vshy" dataset.

```
vshy1['Open_diff'] = vshy1['Open'].diff()
vshy1['Returns'] = vshy1['Close'].pct_change()
vshy1['Returns'] = vshy1['Returns']*100
vshy1['MA7'] = vshy1['Returns'].rolling(window=7).mean().pct_change()
vshy1['MA11'] = vshy1['Returns'].rolling(window=11).mean().pct_change()
vshy1['MA21'] = vshy1['Returns'].rolling(window=21).mean().pct_change()
vshy1['MA31'] = vshy1['Returns'].rolling(window=31).mean().pct_change()
vshy1['VA7'] = vshy1['Close'].rolling(window=5).std()
vshy1['VA11'] = vshy1['Close'].rolling(window=11).std()
vshy1['VA21'] = vshy1['Close'].rolling(window=21).std()
vshy1['VA31'] = vshy1['Close'].rolling(window=31).std()
vshy1['GARCH_vol'] = garch_result.conditional_volatility

vshy1['Volatility_Regime1'] = (vshy1['GARCH_vol'] >
vshy1['GARCH_vol'].rolling(window=5).median()).astype(int)

vshy1['Volatility_Regime2'] = (vshy1['GARCH_vol'] >
vshy1['GARCH_vol'].rolling(window=10).median()).astype(int)

vshy1['Volatility_Regime3'] = (vshy1['GARCH_vol'] >
vshy1['GARCH_vol'].rolling(window=15).median()).astype(int)

vshy1['Volatility_Regime4'] = (vshy1['GARCH_vol'] >
vshy1['GARCH_vol'].rolling(window=25).median()).astype(int)

vshy1['Noise1'] = vshy1['Close'] - vshy1['Close'].rolling(window=3).mean()
vshy1['Garch2'] = vshy1['GARCH_vol'].astype(int).rolling(window=5).max()
vshy1['Garch3'] = vshy1['GARCH_vol'].astype(int).rolling(window=5).min()
vshy1['Garch4'] = vshy1['GARCH_vol'].rolling(window=10).max()
vshy1['Garch5'] = vshy1['GARCH_vol'].rolling(window=10).min()
vshy1['GARCH_residuals3'] = vshy1['Returns']-vshy1['GARCH_vol'].shift(3)
vshy1['GARCH_residuals10'] = vshy1['Returns']-vshy1['GARCH_vol'].shift(10)
vshy1['GARCH_residuals20'] = vshy1['Returns']-vshy1['GARCH_vol'].shift(20)
vshy1['GARCH_residuals30'] = vshy1['Returns']-vshy1['GARCH_vol'].shift(30)
```

2. Generation of Target Feature i.e. 25 days ahead return:

We have denoised the series to smoothen it.

```
vshy1['Return+25'] = vshy1['Close'].shift(-25) / vshy1['Close'] - 1
```

```
vshy1['Return+25'] = vshy1['Return+25']*100
```

```
vshy1['Return+25_Denoised'] = vshy1['Return+25'].ewm(span=5, adjust=False).mean()
```

```
vshy1['Noise'] = vshy1['Return+25'] - vshy1['Return+25_Denoised']
```

```
vshy1["Output"] = vshy1['Return+25_Denoised']
```

Input features are as follows:

'Open', 'Open_diff', 'Returns',

'MA7', 'MA11', 'MA21', 'MA31',

'VA7', 'VA11', 'VA21', 'VA31',

'GARCH_vol',

'Volatility_Regime1', 'Volatility_Regime2', 'Volatility_Regime3', 'Volatility_Regime4',

'Noise1',

'GARCH2', 'GARCH3', 'GARCH4', 'GARCH5',

'GARCH_residuals10', 'GARCH_residuals3', 'GARCH_residuals20', 'GARCH_residuals30',

3. Train-Test split:

We divided the dataset into train , test and validation set, The validation set consists of 30% of training set

```
vtrainshy1 = vshy1.loc['2010-01-01':'2017-12-31']
```

```
vtestshy1 = vshy1.loc['2018-01-01':'2023-12-30']
```

4. Scaling and subsequence of series: We scaled the input features, then created subsequent arrays of 25 window size for each record in the dataset and then created a new dataset for the model to be trained.

Individual RNN models like CNN or LSTM were not performing well the financial data se used a hybrid model CNN+LSTM+with attention

5. Model Architecture:

Local feature extraction (short-term fluctuations)

```
Conv1D(32, kernel_size=3, padding='causal', activation='tanh')(inputs)
```

```
BatchNormalization()(x)
```

```
Dropout(0.2)(x)
```

```
Conv1D(32, kernel_size=3, padding='causal', activation='tanh')(x)
```

```
BatchNormalization()(x)
```

```
Dropout(0.2)(x)
```

Group Number: 8429

Long-term memory (trend)

```
LSTM(64, return_sequences=True)(x)
LSTM(128, return_sequences=True)(x)
```

Transformer-like Attention Block (for importance weighting)

```
MultiHeadAttention(num_heads=4, key_dim=16)(x, x, x)
Add()([x, attn])
LayerNormalization()(attn)
```

More dense layers to learn complex mapping

```
GlobalAveragePooling1D()(attn)
Dense(128, activation='tanh')(x)
Dropout(0.4)(x)
Dense(64, activation='tanh')(x)
Dropout(0.3)(x)
output = Dense(1, activation='linear')(x) # Predict 25-day return
model = Model(inputs=inputs, outputs=output)
model.compile(optimizer='adam', loss=fluctuation_loss, metrics=['mae'])
```

6. Calculate R-Squared and Mean Absolute error on test data**7. Plotting of predictions vs Actuals****2.1 Equity → SPDR S&P500 ETF. Symbol: “SPY”**

We created similar features for every asset ETF and used a similar hybrid model. In this case only for SPY we denoised the Return+25 series and created two columns Trend and Noise.

Where $SPY_Noise = (Return_{+25})_{series} - \text{Denoised SPY Series}$

Multi output prediction: we will try to predict both the 25 days ahead trend and noise of returns.

Our Target: output: [Noise,Trend]

So our final prediction = prediction_trend +prediction_final

Best result at epoch 4

Train Data:

Noise_Output_loss: 0.0462 - Trend_Output_loss: 0.0768 - loss: 0.0523 -

Validation Data:

val_Noise_Output_loss: 0.0090 - val_Trend_Output_loss: 0.0268 - val_loss: 0.0159

In the rest of the assets we directly predicted the denoised series.

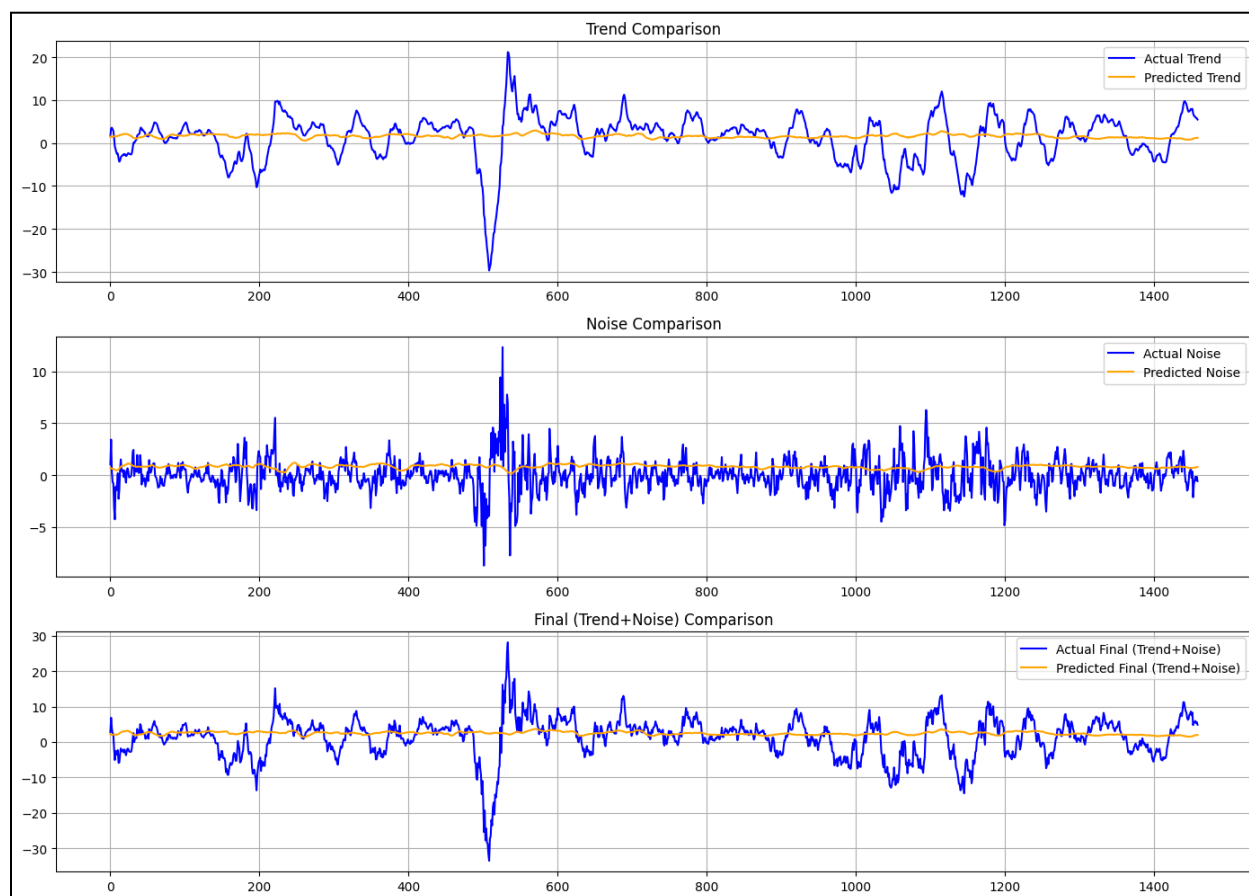


Test Data

Trend: $R^2 = 0.0380$, MAE = 4.2189

Noise: $R^2 = -0.0544$, MAE = 1.1625

Final: $R^2 = -0.0021$, MAE = 4.5531



2.2 Fixed Income → iShares 20+ Year Treasury Bond ETF. Symbol: “TLT”

Best Results at epoch 13:

Train Data - mean_absolute_error: 0.4054

Validation Data- val_mean_absolute_error: 0.2116

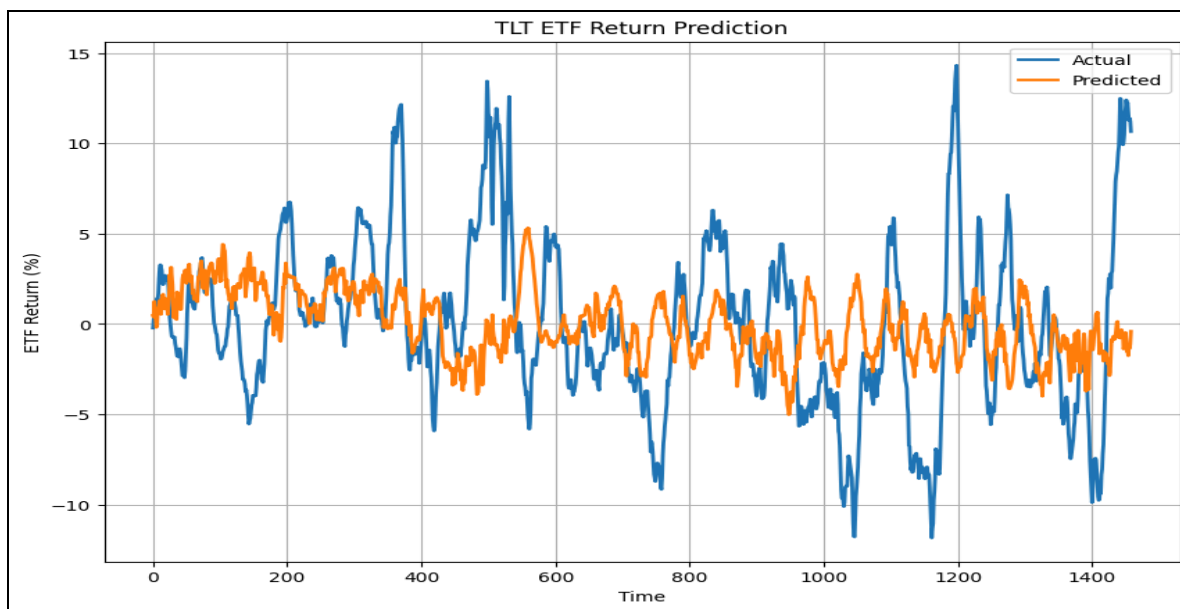


Test Data:

R-squared: -0.1155

MAE: 3.7497

	Date	Pred RNN	Actual_values
0	2018-02-07	0.482192	-0.225878
1	2018-02-08	0.378814	0.231808
2	2018-02-09	1.253322	0.643713
3	2018-02-12	0.742173	0.634522
4	2018-02-13	0.278721	0.515513



2.3 Cash-like → iShares 1-3 Year Treasury Bond ETF. Symbol: "SHY"

Best result on epoch 12

Train data - mean_absolute_error: 0.3577

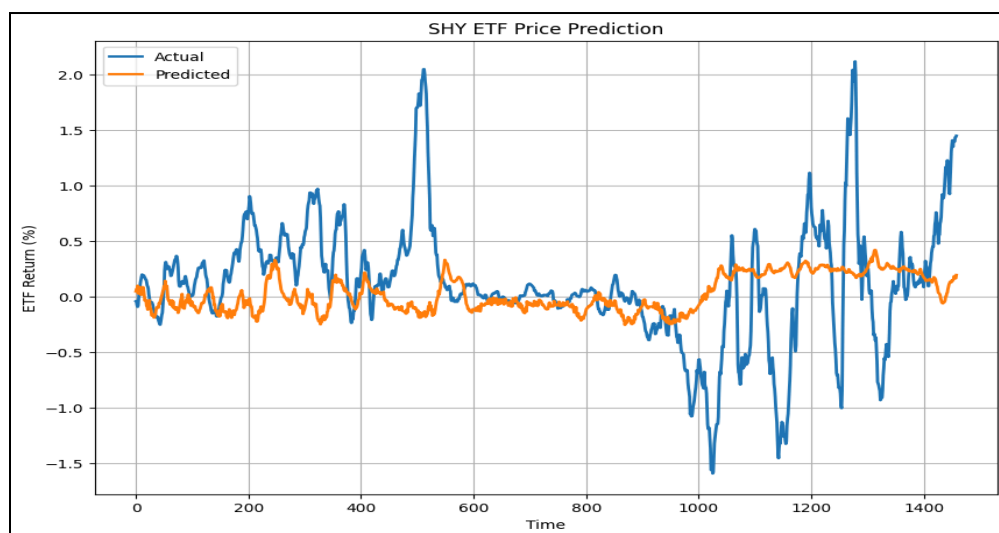
Validation Data - val_mean_absolute_error: 0.2397



Test Data

R-squared: -0.1554

Mean Absolute Error (MAE): 0.4030



GROUP WORK PROJECT # 2
Group Number: 8429

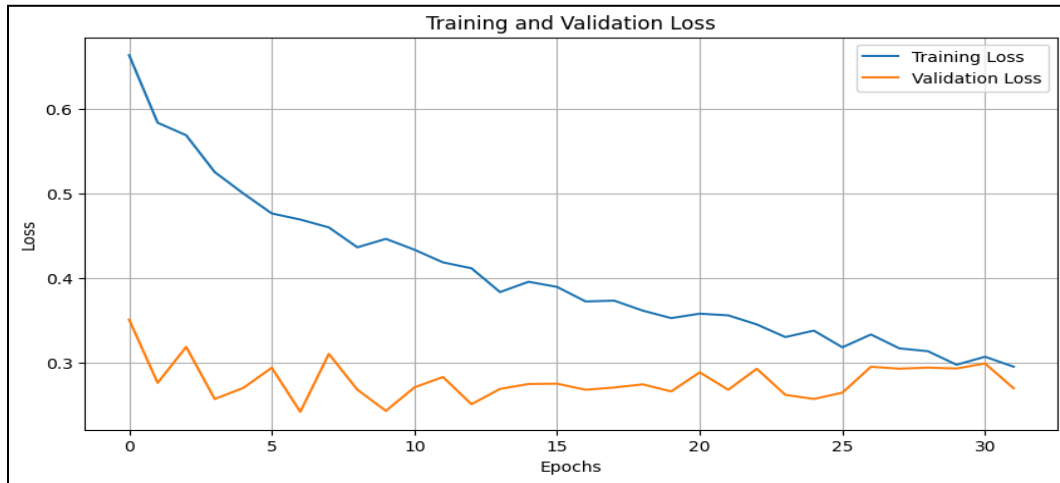
MScFE 642: Deep Learning for Finance

	Date	Pred RNN	Actual_values
0	2018-02-07	0.047481	-0.039770
1	2018-02-08	0.067993	-0.044022
2	2018-02-09	0.068988	-0.074762
3	2018-02-12	0.100214	-0.091298
4	2018-02-13	0.030717	-0.078375

2.4 Precious metals (Gold) ✕ SPDR Gold Shares. Symbol: "GLD"

Train Data - mean_absolute_error: 0.4662

Validation Data - val_mean_absolute_error: 0.2413

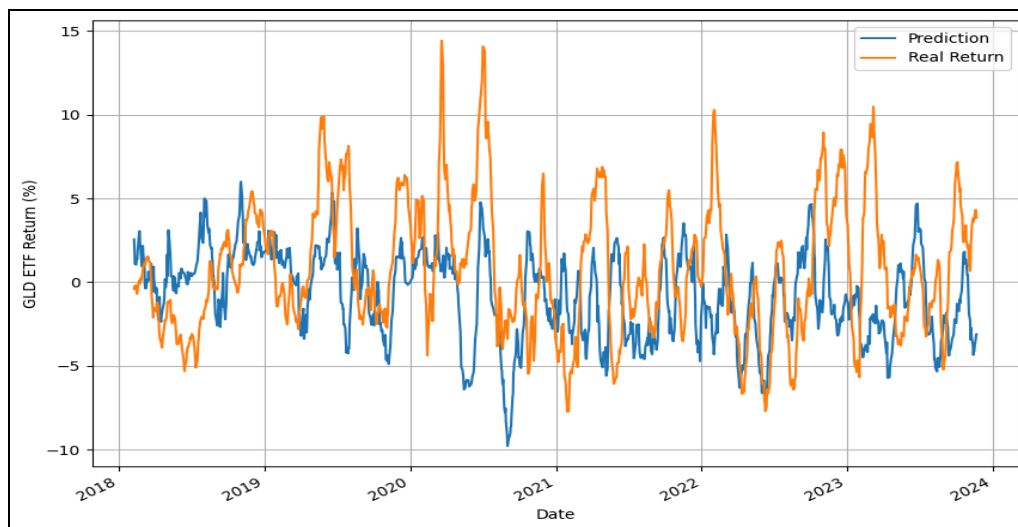


Test Data

R-squared: -0.4312

Mean Absolute Error (MAE): 3.7118

	Date	Pred RNN	Actual_values
0	2018-02-07	2.560869	-0.407712
1	2018-02-08	2.058192	-0.373159
2	2018-02-09	1.089364	-0.222055
3	2018-02-12	1.031904	-0.429871
4	2018-02-13	1.224508	-0.180827



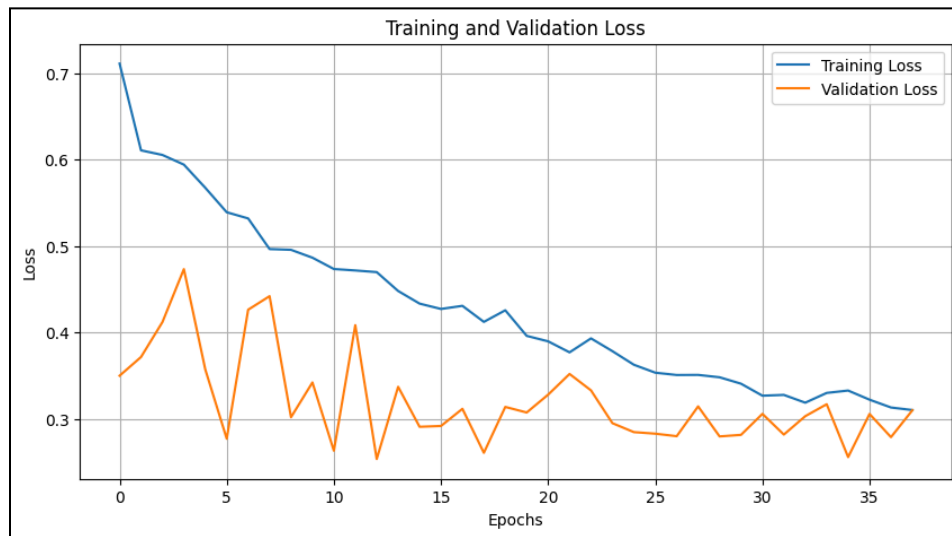
The predictions are able to follow the trend and moderate seasonality but could not keep pace with high fluctuations.

2.5 Crude Oil → Invesco DB Oil Fund. Symbol: “DBO”

Best result at epoch 13

Train Data - mean_absolute_error: 0.4676

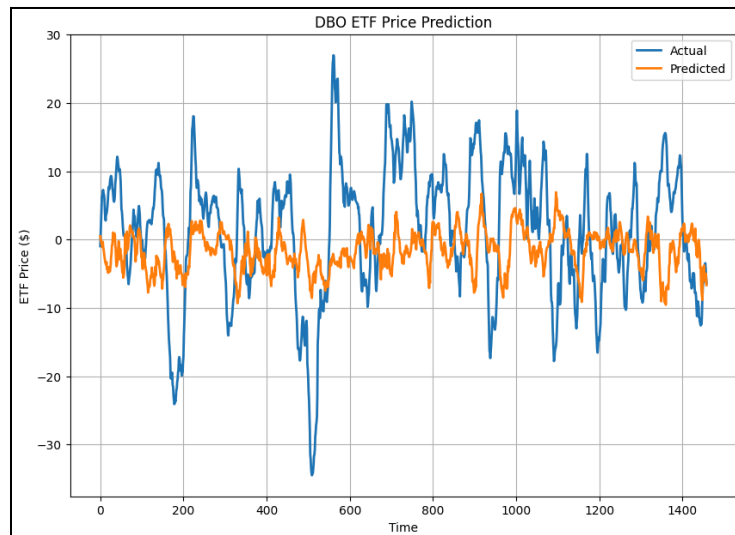
Validation Data- val_mean_absolute_error: 0.2538



Test Data

R-squared: -0.1614

Mean Absolute Error (MAE): 8.4370



The predictions can follow trends and with a return of +/-10% but could not cope with fluctuations higher than that.

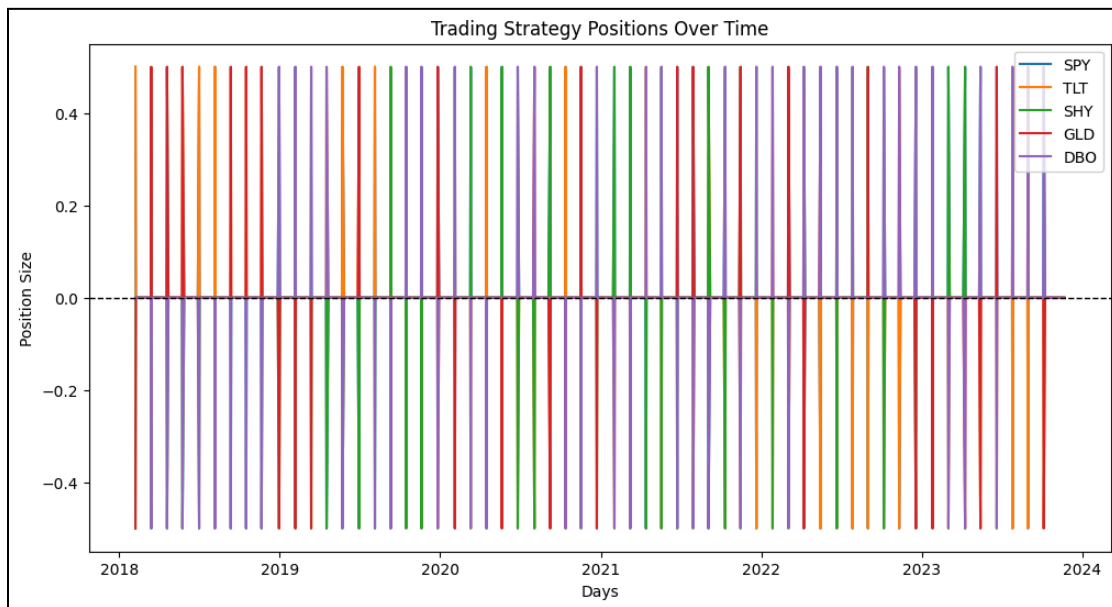
Comparison of Model Performances:**Predictions:**

	Date	Pred RNN(SPY)	Pred RNN(TLT)	Pred RNN(SHY)	Pred RNN(GLD)	Pred RNN(DBO)
0	2018-02-07	2.738462	0.482192	0.047481	2.560869	0.486494
1	2018-02-08	6.857995	0.378814	0.067993	2.058192	-0.002431
2	2018-02-09	3.852119	1.253322	0.068988	1.089364	-0.320872
3	2018-02-12	2.523211	0.742173	0.100214	1.031904	-0.620664
4	2018-02-13	2.072563	0.278721	0.030717	1.224508	-0.782940

Actuals:

	Date	Actual_valu es SPY	Actual_valu es TLT	Actual_valu es SHY	Actual_valu es GLD	Actual_valu es DBO
0	2018-02-07	2.738462	-0.225878	-0.039770	-0.407712	-0.971837
1	2018-02-08	6.857995	0.231808	-0.044022	-0.373159	1.005811
2	2018-02-09	3.852119	0.643713	-0.074762	-0.222055	2.855244
3	2018-02-12	2.523211	0.634522	-0.091298	-0.429871	4.475214
4	2018-02-13	2.072563	0.515514	-0.078375	-0.180827	6.366073

SPY and Gold are favorites among others

Trading Strategy: Long vs Short Positions**Trading Strategy 1**

The vertical bar represents long(+0.5) and short(=-0.5) positions.

We can see that the position changes at regular intervals, also one position does not continue for long which may impact higher returns.

The top 2 assets are assigned positive weight (long) and bottom two assets are assigned negative weights (short). I.e. +/-0.5. Some assets do not appear in certain periods because they were neither on top 2 or bottom 2 .

Without Shy Cum. returns:

Date	Cumulative Return
2018-02-07	3.892082e+00
2018-02-08	3.892082e+00
2018-02-09	3.892082e+00
2018-02-12	3.892082e+00
2018-02-13	3.892082e+00
...	...
2023-11-16	1.552286e+46
2023-11-17	1.552286e+46

2023-11-20	1.552286e+46
2023-11-21	1.552286e+46
2023-11-22	1.552286e+46

Trading Strategy 2

We adjusted our long and short strategy by making one more change:

We advised going Long for Assets where predicted return > 0 and Short for Assets where predicted return < 0 . This helps avoid weak signals.

Also picking top2 (for long) or bottom 2 assets (for shorts)

Giving equal weights to short and long positions ± 1 each.

Cumulative Return

Date	Cumulative Return
2018-02-07	2.960047e+00
2018-02-08	2.960047e+00
2018-02-09	2.960047e+00
2018-02-12	2.960047e+00
2018-02-13	2.960047e+00
...	...
2023-11-16	-9.277556e+52
2023-11-17	-9.277556e+52
2023-11-20	-9.277556e+52
2023-11-21	-9.277556e+52
2023-11-22	-9.277556e+52

Trading Strategy 2 has higher cumulative returns than Strategy 1

Trading Strategy 3

Rather than just following long and short positions based on top 2 and bottom 2 assets, We tried to adjust our strategy by implementing following changes:

- Use `min_trend_threshold = 0.02`: i.e. Minimum predicted return required to trade.
- `min_volatility_threshold = 0.01`: Assets below this 25 day rolling volatility are ignored.
- `early_exit_threshold = 0.05`: **Exit early** if return reaches $\pm 5\%$ before 25 days otherwise default rebalancing period as 25 days

Date	Cumulative Return
2018-02-07	1.480024
2018-02-08	9.541531
2018-02-09	18.582964
2018-02-12	6.577646
2018-02-13	12.088947

We found Trading Strategy 3 has the highest cumulative returns.

Long position aligns with uptrend and short position aligns with downtrend.

If the markets are trending, long term holding may prove to be beneficial.

If the markets are mean reverting in a short term period, then frequent rebalancing may be the strategy..Short term trends may cause early exits thus limiting gains .

Some ETF do not have high returns and move around zero or $\pm 5\%$ and then reverse the trend.

In such cases we can check by using a momentum or volatility filter, or adjust the balancing period dynamically and not wait for 25 days for e.g. if an asset goes up by 7% in the next 20 days and then drops 4% then closes the position before 25 days.

Stop loss can be used to safeguard against a threshold level once it crosses above or below the level.Taking early exit may minimize the losses.

Cumulative Return:

For Long positions (+0.5): If the predicted return is positive, we take a long position (e.g., +0.5 weight), meaning we profit when the asset's return is positive.

For Short position (-0.5): If the predicted return is negative, we take a short position (e.g., -0.5 weight), meaning we profit when the asset's return is negative.



In the above fig. We did not include SHY, Below has less returns with SHY



The plot remains flat for most of the period due to low value of cumulative return and later stage it grows in magnitude. A lack of diversification may lead to concentrated risk because if some ETF has downtrend then it should be balanced by some uptrend ETF in the portfolio.

There could be ineffective trading signals during most of the period which could result in no trading or loss.

Trading Strategy 4: Buy and Hold till the end (with no rebalancing)

Date	Portfolio_Value	Daily_Return	Cumulative_Return
2018-02-07	1.000000	0.000000	0.000000
2018-02-08	2.535673	1.535673	1.535673
2018-02-09	9.851505	2.885163	8.851505
2018-02-12	36.212768	2.675861	35.212768
2018-02-13	146.121352	3.035078	145.121352

Limitations: does not react to market conditions

Trading Strategy 5: Monthly Rebalancing

Date	Portfolio_Value	Daily_Return	Cumulative_Return
2018-02-07	1.218640	0.000000	0.000000
2018-02-08	6.335127	4.198522	4.198522
2018-02-09	29.092216	3.592207	22.872692
2018-02-12	101.377253	2.484686	82.188848
2018-02-13	311.309460	2.070802	254.456470

Trading Strategy 6: Equally weighted

Taking long (for positive returns) or short position (for negative returns) on all ETFs , not just select two (as chosen earlier)

Date	Portfolio_Value	Daily_Return	Cumulative_Return
2018-02-07	1.101011	0.101011	0.101011
2018-02-08	2.860620	1.598177	1.860620
2018-02-09	5.925158	1.071284	4.925158
2018-02-12	9.896466	0.670245	8.896466
2018-02-13	15.432169	0.559362	14.432169

BackTesting:

Dynamic rebalancing i.e. Monthly Rebalancing and Early exit with threshold advisory gives better cumulative return rather than just buy and hold or equally weighted portfolio..

STEP 3

b)

In-Sample Loss:

The in-sample loss values are consistently low, indicating that the model fits the training data well.

Out-Sample Loss:

The out-of-sample loss values remain close to the in-sample losses, suggesting that the model generalizes reasonably well to unseen data.

The relatively small gap between in-sample and out-of-sample losses across all cases suggests that the model does not suffer from severe overfitting.

Overall, the model demonstrates solid predictive power with relatively stable performance across training and testing datasets.

c)

Buy-and-hold return of an Equally weighted portfolio of the 5 asset classes

Final Portfolio Return - -0.00405

Total Buy-and-Hold Return - 0.00102

d)

i)

The final portfolio return is negative (-0.00579), which is concerning, especially when compared to the buy-and-hold return (0.00103). This indicates that the model-driven strategy underperformed relative to a simple passive holding approach. The Sharpe Ratio of 0.2346 suggests that the portfolio is generating low excess returns relative to its risk. The buy-and-hold strategy outperformed the model-based portfolio. This implies that the trading strategy either failed to capture meaningful patterns or incurred excessive transaction costs that eroded profits.

ii)

Comparing the multi-output model's portfolio of 5 asset classes with single output model's equally weighted portfolio, we find that the single output model performed much better in comparison to multi-output model

STEP 4

A comparison of multiple-output and single-output designs reveals considerable variations in predictability and back-testing performance. The single output models, which predicted 25-day ahead returns independently for each ETF, provided greater optimum accuracies for capturing the distinctive dynamics of each asset class than this, showing that a single outlier model just worked perfectly for term movements.

In contrast, the multiple yield model from Step 3 sought to estimate the returns of the five ETFs concurrently has poor returns. This implies that the multiple-yield model of global market dynamics, even though it could capture, failed to distinguish and take advantage of the distinctive characteristics of each asset class, leading to poor performance in the post-test phase. Single output models were more successful in leveraging unique asset behavior, making it a short-term trade. The performance of the multi-execution model has exceeded the methods, highlighting the issue of balancing information sharing across all properties without losing attention on critical subtleties.

Comparison summary:

1. Single-Output Models :
 - Models that projected 25-day forward returns for each ETF had greater accuracy in reflecting the distinct dynamics of each asset class. These models successfully utilized the unique characteristics of individual assets, as seen by their improved performance.
2. Multi-Output Model :
 - The multi-output model, which sought to forecast the returns of all five ETFs at the same time performed not so good. This shows that, while the model was able to capture certain broad market trends, it struggled to discriminate and capitalize on each ETF's specific qualities. The total performance was much lower than the single-output models.

References

- Moghar, Adil and Hamiche, Mhamed. Stock Market Prediction Using LSTM Recurrent Neural Network, Procedia Computer Science, Volume 170, 2020
- G. Tsang, J. Deng and X. Xie, "Recurrent Neural Networks for Financial Time-Series Modelling. 2018 24th International Conference on Pattern Recognition (ICPR), Beijing, China, 2018
- Mienye ID, Swart TG, Obaido G. Recurrent Neural Networks: A Comprehensive Review of Architectures, Variants, and Applications. Information. 2024
- Kondeti Poojitha, L. Venkata Krishna Rao, Koppula Thirumala Dhilip Kumar, Koneru Chaitanya Chandra, Maganti Varsha, "Predicting Stock Price Movements in Volatile Markets: A Multi-Model Fusion Approach", 2024